



Projeto Integrador – Mini Shell com Chamadas ao Sistema
Prof. Dra. Larissa Barbosa Leôncio Pinheiro

OBJETIVO PRINCIPAL DO PROJETO:

Desenvolver um mini interpretador de comandos (shell) que simula a execução de comandos em um terminal Linux, explorando **chamadas ao sistema**. O projeto será implementado em **C ou Python**.

O shell é um programa que permite ao usuário interagir com o sistema operacional por meio de comandos digitados. Ele interpreta esses comandos e os executa, seja diretamente ou por meio de programas externos. Um mini shell é uma versão simplificada de interpretador de comandos, como bash ou, zsh. É especialmente útil para fins educacionais, pois proporciona prática com conceitos fundamentais de sistemas operacionais, como a criação de processos, manipulação de entrada e saída padrão, e controle da execução de programas.

Um mini shell normalmente implementa funcionalidades como:

- Prompt (>) – Indica ao usuário que o shell está pronto para receber comandos.
- Leitura de comandos – Captura o que o usuário digitou, incluindo argumentos.
- Criação de processos – Usa `fork()` (ou equivalente) para executar o comando em um processo filho.
- Execução de comandos – Usa `exec()` para substituir o processo filho pelo comando digitado (ls, echo, cat).
- Espera pela execução – Usa `wait()` para que o processo pai aguarde o término do processo filho.
- Encerramento com `exit` – Permite ao usuário finalizar o shell de forma controlada.

DESCRIÇÃO DAS TAREFAS:

Deve ser implementado um programa que:

- Exiba um prompt interativo ao usuário (>)
- Leia um comando digitado, inclusive com argumento (ex. ls -l, echo Ola mundo).
- Crie um processo filho usando `fork()` ou `os.fork()`.
- O processo filho deve executar o comando `execvp()` ou `os.execvp()`.
- O processo pai deve aguardar o término do filho usando `wait()` ou `os.wait()`.
- O programa deve continuar executando até que o usuário digite `exit`.
- Exibe mensagens adequadas caso o comando não seja encontrado ou ocorra algum erro.

REQUISITOS MÍNIMOS OBRIGATÓRIOS:

- Usar `fork()`, `execvp()`, `wait()`, `read()` e `write()`.
- Tratar entrada padrão usando `read()`.
- Executar no mínimo três comandos diferentes corretamente e com argumentos (ls, cat, echo).
- Tratar erros com mensagens amigáveis (ex. comando não encontrado).
- Modularização mínima: funções separadas para leitura/parsing, execução e controle de loop principal.

ENTREGA:

- Código fonte do mini shell
- Um arquivo **README** contendo:
 - Como compilar e rodar.
 - Quais chamadas ao sistema foram utilizadas.
 - Exemplos de comandos testados e suas saídas.
 - Limitações conhecidas da implementação.

- Vídeo curto demonstrando o uso do shell (máximo 5 min).

APRESENTAÇÃO:

A apresentação deve abordar:

- Introdução e contexto do projeto.
- Conceitos de chamadas ao sistema aplicados.
- Explicação da estrutura e funcionamento do código.
- Demonstração prática do mini shell.
- Dificuldades enfrentadas e aprendizados.
- Conclusão e espaço para perguntas.

ANEXOS:

Aspecto	Versão em C	Versão em Python
Criação de processo	<code>`fork()`</code>	<code>`os.fork()`</code>
Execução de comando	<code>`execvp()`</code>	<code>`os.execvp()`</code>
Espera pelo filho	<code>`wait()`</code>	<code>`os.wait()`</code>
Leitura de entrada	<code>`read()`</code>	<code>`input()`</code> (ou <code>`os.read()`</code> para bônus)
Saída de dados	<code>`write()`</code>	<code>`print()`</code> (ou <code>`os.write()`</code> para bônus)
Controle de erros	<code>`perror()`</code> e códigos de erro	<code>`try/except`</code> e <code>`os.strerror()`</code>
Redirecionamento e pipes	<code>`dup2()`</code> , <code>`pipe()`</code>	<code>`os.pipe()`</code> , <code>`os.dup2()`</code> (desafios avançados)

Recomenda-se usar `os.read()` e `os.write()` ao invés de `input()` e `print()`.